

file d.out p.out s.out

ELF files and analysis

Generated object files, executables and libraries in Linux are known as ELF (executable and linkable format) files. To analyse ELF files you can use a few dissection tools listed below. Even though MingW generates Windows PE format, the effect of these tools (except *readelf*) will be the same.

nm: This is used to print a symbol table from an object file or executable. All the global variable and function names seen by the linker are known as symbols.

Here, you can observe that *sum* and *square* are undefined in *test.o* and defined in *sum.o*, *sqr.o*. A lot of runtime support symbols are added in *all.out*, as follows:

```
nm test.o sum.o sqr.o all.out
```

objdump: This is used for the dissection of ELF files in terms of disassembly, symbols, section details, etc.

```
objdump -d test.o #disassemble ELF files
objdump -t test.o #displays symbol info
objdump -x test.o #displays all headers
objdump -S test.o #intermixed source and assembly, code has
to be compiled with debug support using -g option
```

readelf: This is used to provide meta information about ELF files (Linux only).

```
readelf -h test.o all.out #Provides ELF header details
like magic number, target
#architecture,endianness, ABI, etc.
readelf -a all.out
#Provides all headers and section details
```

To know the size of the text, data, bss sections and the total, we can use the *size* command, as follows:

```
size test.o all.out
```

Last but not the least, the *file* utility mentioned earlier is used to know each type of file. Based on the magic number in the header part of *file*, it can distinguish between object files, executables, static libraries and dynamic libraries.

```
file *.o all.out libsamle.a libsamle.so
```

Sections of a program/process

An actively loaded program can have various sections like code (*.text*), initialised data (*.data*), uninitialised data (*.bss*), read-only data (*.rodata*) and stack. To see the applicable sections of symbols in a code, let's add a few global variables and functions, as follows. Compile as object file and see the symbol table using *nm* or *objdump*. Then check the

indicated symbol states:

```
int d1=10;           //D
int c;               //C
static int d2=10;     //d
static int b1;        //b
const int r1=10;      //R
const int r2;         //r
void foo() { }        //T
static void bar() { } //t
```

The explanation of the above code is given below.

T, t: *.text*
D, d: *.data*
R, r: *.rodata*
C: Common symbols, will be merged into *.bss* on linking
B: *.bss*
W, w: Weak symbols
T, R, D: Indicates eligible for external linkage
t, d, r: Indicates restricted for internal linkage only

You may wonder about the invisibility of local variables, which are not considered as symbols. These are converted into offsets with respect to the stack frame using a stack pointer, frame pointer registers (ESP, EBP in x86) thus not seen by linker, which is clear from disassembly using *objdump* also.

Weak symbols are almost similar to declarations, but don't cause a linker error if they are not defined by the user. These are useful for interrupt handlers, exception handlers or any event handlers that can be aliased to default handlers. Users can override these with their own custom handlers. Let's add this code in *test.c* to check for weak symbols quickly and check *nm* output.

```
void f1() {
    #default code for f1
}
void f1() __attribute__((weak)); #f1 can be redefined
void f2_default() {
    #default code for f2
}
void f2() __attribute__((weak,alias("f2_default")));
#f2 can be redefined
```

f1 or *f2* can be redefined by the user only once as strong symbols in other translation units, or can be redefined as weak symbols multiple times subsequently. *f2_default* will be invoked if *f2* is not redefined which is aliased to *f2*. If a weak function is not aliased to a strong function, not having default code in the same translation unit or not redefined further by user causes runtime error.

Function overloading and name demangling

Let's look at a few ways in which the tools are used on C++